THESIS TITLE

A PREPRINT

Knud Rasmus Sønderbek

April 27, 2026

ABSTRACT

Some super cool abstract of all my findings

1 Temp intro

Introduction A central goal of mechanistic interpretability is to decompose a neural network's parameters into a small number of meaningful units, atoms, each of which corresponds to something the model is doing. The hope is that a network's computation, however dense and distributed in its raw form, can be expressed as a sum of simpler mechanisms that admit human-readable descriptions.

A recent line of work shifts the object of decomposition from activations to parameters, decomposing a network's weights into a sum of simpler components, each taken to correspond to a concrete atom of computation. Methods guided by the principle of minimum description length aim to recover components that are faithful to the original weights, minimal in the number required per input, and individually as simple as possible. Stochastic Parameter Decomposition (SPD) [Bus+25] is one such method, decomposing networks into rank-one subcomponents and learning a per-component importance function that estimates each atom's causal contribution to a given input.

The Open Problems in Mechanistic Interpretability programme [Sha+25] identifies the question of how best to carve a network at its joints as a central open problem of the field. Whether a parameter decomposition recovers the computational primitives the model actually uses depends on what the ground-truth decomposition is taken to be, and the carving question is the question of which units to take as ground truth. For models with independent features the answer is comparatively unambiguous: one atom per feature. The question becomes substantially harder when the features themselves are dependent, a regime that is empirically attested in real models. Engels et al. [Eng+24] develop a rigorous notion of irreducible multi-dimensional features, defined as features whose joint distribution can be split neither into independent components nor into disjoint mixture components, and show that language models represent inherently cyclical concepts, including days of the week and months of the year, as two-dimensional irreducible features whose individual coordinates are not independent units but components of a single object the model manipulates together. Networks that represent such features admit multiple decompositions that are equally faithful to the weights but differ in the level at which features are separated. A network that computes the volume of a cube could equally faithfully be decomposed into separate components that each compute one of the side lengths, or into a single component that computes the volume directly. The first is a feature-level decomposition: one atom per individual input direction, exposing the internal structure of the computation but leaving the grouping of features implicit, recoverable only by inspecting which atoms co-activate. The second is a structural-level decomposition: one atom per irreducible structure, exposing the grouping directly but treating the internal computation as a single unit. Each level loses something the other preserves.

The present work extends SPD by examining its behaviour on dependent feature pairs and larger structures, configurations in which the multi-dimensional superposition hypothesis posits a structural-level unit. Here SPD is shown to recover both feature-level and structural-level decompositions, and the conditions distinguishing the two outcomes are characterised. Building on this, we introduce softmax-and-attention competition between subcomponents, making the importance calculation relational, and show that the extension allows SPD to recover decompositions that respect the multi-dimensional superposition hypothesis. We additionally introduce the Toy Model of Geometry, a small model that computes the volume of a simplex, to examine how this method performs on a more complex structure.

2 Old Introduction - may be useful for related work.

Neural networks continue to achieve remarkable performance across domains ranging from medical diagnosis to autonomous systems. This increase in performance has, however, come with a corresponding increase in complexity, making these systems less interpretable to those who build and deploy them. As neural networks enter high-stakes environments, where errors carry legal, financial, or human cost, interpretability becomes a necessity. The concern is not merely technical but ethical: a fundamental principle of accountability holds that the creator of a tool can only be held responsible for its outcomes to the extent that those outcomes could have been reasonably foreseen. A system whose internal reasoning cannot be understood is, by definition, one whose failures cannot be anticipated.

Mechanistic interpretability aims to reverse-engineer the internal computations of neural networks into their simplest forms, such that they can be expressed as humanly interpretable algorithms.
Many of the current popular methods within this space, such as sparse autoencoders [Cun+23], circuit analysis, and probing, primarily operate in activation space. However, each of these methods comes with its own set of limitations. Sparse autoencoders treat features as independent directions and discard the geometric structure between them, despite growing evidence that this structure carries meaningful information (Leask et al., 2025; Mendel, 2024; Engels et al., 2024). Moreover, sparse autoencoders are trained to reconstruct activations and do not to explicitly identify the computational units that produced them. [Lea+25; Cha+25] The resulting decomposition may be functionally equivalent at a given layer, while not resembling the mechanisms that the network employs. Circuit-level methods face the deeper issue that neural networks are densely connected, meaning any given output can be equally well attributed to a multitude of causal pathways, leaving the choice of circuit underdetermined.

A more recent line of work shifts the object of decomposition from activations to parameters. More concretely, these methods aim to decompose a network's weights into a sum of simpler components, each of which corresponds to a concrete "mechanism". This reframing sidesteps several of the issues above: mechanisms are no longer constrained to specific architectural components, such as neurons or attention heads, and can instead be distributed across multiple architectural components. Moreover, they decompose the network into functional units rather than representational directions, which may further enhance their interpretability.
Recently proposed methods guided by the principle of minimum description length have shown promising results in this direction, aiming to decompose a network into parameter components that are faithful to the original weights, minimal in the number required per input, and individually as simple as possible.
One such method, Stochastic Parameter Decomposition (Bushnaq et al., 2025), decomposes networks into rank-one subcomponents. Since not all components are relevant for every input, the method learns a function that estimates each component's causal importance for a given input. However, each component's importance is calculated independently, relying only on its own activation. As noted in the original work, this is likely too restrictive for non-toy models, as it prevents each component's importance from being informed by the state of other components. This work replaces the independent importance functions with a graph neural network that determines each component's importance conditioned on its relationships to all other components, while preserving explainability through its generalised additive structure. Additionally the use of an additive importance function, generalizes the decomposition to k-dimensional components. This generalizsation is motivated by the growing evidence, that transformer models, learn features which occupy irreducible multidimensional subspaces.

[Bra+25]

3 Related work

-Open problems in mechanistic interpretability - SPD - SPD on transformers - APD - Engels et al. -Activation space decompositions - Circuits, SAE, Linear representation hypothesis Superposition in general? Strong feature hypothesis

4 Method

4.1 Background on APD and SPD

The initial inspiration for shifting the focus from activations to parameters came from Attribution-based Parameter Decomposition (APD) [Bra+25]. APD decomposes the weights of a network into a set of parameter components: vectors in parameter space that satisfy three desirable properties.

Let $f(x, W)$ be a neural network with weight matrices $W = \{W^1, \ldots, W^L\}$. A set of parameter components should satisfy the following properties:

- **Faithfulness**: The parameter components should sum to the parameters of the original network.
- **Minimality**: As few parameter components as possible should be required to replicate the network's output on any given input.
- **Simplicity**: Each component should span as few weight matrices and as few ranks as possible.

If a set of parameter components satisfies these properties, they are said to comprise the network's mechanisms [Bra+25].

However, APD suffers from several practical limitations. Each parameter component is a full vector in parameter space $\mathbb{R}^{|\theta|}$, where $|\theta|$ is the total number of parameters in the network, thus increasing the memory cost to $O(\theta \cdot C)$, where C is the number of components, making the method computationally expensive to scale to larger models. APD is also highly sensitive to its top-k hyperparameter, which specifies the expected number of active components per input and must be chosen in advance. Lastly, APD relies on the magnitude of the gradient as a proxy for the causal importance of each component. These gradients have been shown to be poor approximations of the true importance. (cite the papers here).

Stochastic parameter decomposition

Stochastic Parameter Decomposition (SPD) was introduced as a solution to the limitations of APD. SPD decomposes each weight matrix into a sum of rank-one matrices, called subcomponents:

$$W^l \approx \sum_{c=1}^{C} \vec{u}_c^l \vec{v}_c^{lT}, \quad \text{where} \quad \vec{u} \in \mathbb{R}^{d_{out}}, \vec{v} \in \mathbb{R}^{d_{in}}$$

The number of subcomponents C is a hyperparameter that may be chosen to be larger than the rank of W^l enabling the method to learn features in superposition [BBS25].

These subcomponents are then trained to be faithful to the original weights, to have as few active subcomponents as possible for any given input, and lastly to reconstruct the original network's output.

4.1.1 Faithfulness

4.1.2 Stochastic reconstruction

4.1.3 Minimality

4.1.4 Limitations

These components are later clustered manually.... -¿ GNaN solves Components are rank 1, many features are multidimensional, dependent components are trained independently

4.2 Irreducibility and natural units of decomposition

Develops Engels et al.'s framework formally. The irreducibility definition, the multi-dimensional superposition hypothesis, and the implication that the natural unit of decomposition is the irreducible feature with its associated V V matrix. This subsection is doing the theoretical work of establishing what a faithful decomposition should target. It does not yet claim SPD fails to target this; it just establishes the target.

4.3 Effective representational rank

The distinction between formal dfi dfi and the effective rank the model uses to represent an irreducible feature. The two coincide for non-degenerate cases (simplex, circle) but come apart for degenerate cases (perfect correlation, where the model can compress to lower rank). The implication: the rank of an SPD subcomponent that hosts an irreducible feature should match the effective representational rank, which is at most dfi dfi but can be less. This subsection sets up both the rank-1 case (compressed representations) and the rank-k case (full representations) as targets the method needs to handle.

4.4 Rank-k subcomponents and relational importance

The framework of Engels et al. described in section 4.2 identifies that these irreducible features are the desired units of decomposition, with internal dimensionality d_{f_i}. SPD's rank-one parameterisation can contain such a feature as

a single atom only when its effective representational rank is one. [1] Otherwise the decomposition must split these features across multiple subcomponents.

To accommodate this, each pair of vectors in SPD's parameterisation is replaced with a pair of matrices:

$$W^l \approx \sum_{c=1}^{C} U_c^l {V_c^l}^\top, \qquad U_c^l \in \mathbb{R}^{d_{\text{out}} \times k}, \quad V_c^l \in \mathbb{R}^{d_{\text{in}} \times k} \tag{1}$$

Each subcomponent $U_c^l {V_c^l}^\top \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is a matrix of rank at most k, and setting $k = 1$ recovers the original SPD formulation.

This definition allows each subcomponent to span up to k rank-one factors but does not require it to do so; mechanisms for encouraging atoms to use less than their full capacity are discussed in Section **??**.

The extension to the importance function follows naturally from the change in parameterisation. The inner activation used to estimate each subcomponent's importance is now a k-dimensional vector:

$$h_c^l(x) = {V_c^l}^\top \vec{a}^l(x) \in \mathbb{R}^k \tag{2}$$

where $\vec{a}^l(x)$ is the activation of layer l for input x.

Lastly we replace the independent importance function with a relational one instead of the original independent one, where each components importance was derived from its own activation, independent of all other subcomponents. The justification for this is shown in Section **??**.

We aim to motivate the importance through a probabilistic interpretation, under which importance becomes the posterior probability that a given subcomponent is responsible for the output, conditioned on the activations of all subcomponents in the same layer:

$$\alpha_c^l(x) = p\big(c \mid h_c^l(x), \{h_{c'}^l(x)\}_{c' \neq c}\big) \tag{3}$$

[2] Importance becomes the probability that subcomponent c owns the input, conditioned on the activations of all subcomponents.

The posterior is realised through a softmax over subcomponents,

$$\alpha_c^l(x) = \frac{\exp s_c^l(x)}{\sum_{c' \in C} \exp s_{c'}^l(x)} \qquad s_c^l(x) = f_{\text{self}}(h_c^l(x)) + f_{\text{agg}}\big(\{h_{c'}^l(x)\}_{c' \neq c}\big) \tag{4}$$

The f_{self} maintains the original signal from c's own activation, while f_{agg} acts as a corrective term that contextualises c against other subcomponents through an attention mechanism.

It is noted, that this attention mechanism has computational cost $O(l \cdot C^2)$, where C is the number of subcomponents per layer and l is the number of layers, leading to scalability concerns at larger sizes.

To circumvent this, we apply fastattention, reducing the complexity to be linear in the number of components. [3].

4.5 SVD reformulation, capacity and per feature ablation

4.5.1 GNAN-based importance functions

In SPD, each importance function γ_c^l is an independent MLP that receives only its own inner activation. Intuitively, however, causal importance is relational: whether a subcomponent can be ablated without affecting the output may well depend on which other subcomponents are active alongside it.

To address this, the independent MLPs are replaced with a Graph Neural Additive Network (GNAN) that can relate each subcomponent to its neighbourhood.

The C subcomponents are treated as nodes in a graph, with node features given by their inner activation vectors $\mathbf{h}_c^l(x) \in \mathbb{R}^k$. The GNAN updates each node's representation through an additive neighbourhood aggregation. For

[1] We Say effective rank, as if the model compresses a 2D direction into a line, then we can actually represnet it

[2] maybe just do all components? Does it matter?

[3] Double check this, fast attention was also used by someo ther paper on SPD and transformers, but applied to the sequence length.

node i in layer l and feature dimension k [4]:

$$[\mathbf{z}_i]_k = \sum_{j=1}^{N} \frac{1}{|\mathcal{L}(j)|} \cdot \rho\left(\frac{1}{1+\text{dist}(j,i)},\ \cos(j,i)\right) \cdot f_k([\mathbf{h}_j]_k)$$

OBS: Implementation ill present today is this:

$$\mathbf{z}_i = \sum_{j=1}^{N} \rho\left(\text{layer distance}, \text{cosine similarity}\right) \cdot f_k\left([\mathbf{h}_j]_k\right)$$

where f_k is a learned function applied to the k-th inner activation of each neighbouring node j; $|\mathcal{L}(j)|$ is the number of nodes in the same layer as j, which normalises the contribution of each layer; $\text{dist}(j,i)$ is the layer distance between nodes j and i[5]; $\cos(j,i)$ is the cosine similarity between their parameter vectors; and ρ is a learned function that maps both the distance kernel and the cosine similarity to a scalar weight.

z_i becomes a vector in $\mathbb{R}^k$, which fits perfectly with the current activation vectors.

A separate MLP is trained to combine the aggregated neighbourhood information and the node's own activation into an importance value in $[0,1]$: Producing a single importance value per subcomponent and passing it through a leaky hard sigmoid, following SPD [BBS25]:

$$g_c^l = \sigma_H(AGG(\mathbf{z}_i^l, \mathbf{h}_i^l))$$

The self-connection through f_{self} (or just the raw activation) ensures that each node retains its own identity after aggregation, mitigating the oversmoothing that can occur in GNNs. {Clever cite here, i.e gnn book}

Alternative formulation, that would work with the next section

After the GNAN update, the per-dimension importance values are computed from the updated representation and passed through a leaky hard sigmoid, following SPD [BBS25]:

$$\mathbf{g}_c^l(x) = \sigma_H\left(\mathbf{h}_c^l(x)\right) \in [0,1]^k$$

in other words, the importance function becomes multidimensional, and each column can be masked independently.

Remarks:
For now the edge set construction is left as the fully connected graph, which remains reasonably computationally expensive for small models. However it could very easily be made more efficient, for other architectures, or by considering only the k-hop neighbourhood of each node, or only considering prior neighbours or later neighbourhoods. Likewise we could store both distances as sparse structures as they are symmetric. $\cos(j,i) = \cos(i,j)$
Remark 2:
The choice of edge set construction and distance function is a very important design choice, e.g using the layer distance on a fully connected graph encodes the same signal for all nods in a layer: For nodes i_1, i_2 in the same layer l:

$$\forall\, j\colon \quad \text{dist}(j,i_1) = \text{dist}(j,i_2) = |k_j - l|$$

and therefore

$$[\mathbf{h}_{i_1}]_k = \sum_{j=1}^{N} \frac{1}{\#\text{dist}_{i_1}(j,i_1)} \cdot \rho\left(\frac{1}{1+|k_j-l|}\right) \cdot f_k([\mathbf{x}_j]_k) = [\mathbf{h}_{i_2}]_k$$

since no term in the sum depends on the identity of i within layer l, only on the layer index l itself.
Additionally this produces a counterproductive signal, the gnan tries to boost or suppress all nodes in a layer together, instead of learning to differentiate between them.

Remark 3:
The choice of distance should probably be reconsidered in general, I have a feeling that the function needs to be from an asymmetric metric space, i.e

$$dist(i,j) \neq dist(j,i)$$

[4] I am very open to alternative equations, i.e perhaps we delete the layer distance inside rho, however it seemed necessary to use cosine similarity, as otherwise the signal was useless as described in some earlier section

[5] The layer distance, could later be signed to differentiate between previous and future layers

Atleast if the intution is if component A in layer k, is highly active, perhaps component B in layer k+1 should likely be inactive, as the other component needs to handle this feature, thus the distance from A to B should be different, as otherwise both are boosted or suppressed together.

Remark 4:
The current experiments are more sensitive to hyperparameters, than the original code seems to be, if the minimality pressure is too low, then all components are active, and doesn't "kill" any of them

Remark 5:
I haven't been able to find any "benefit" in setting k higher, this is ofcourse because hte ground truth of all the models we have are rank 1 independent features, we need to find a scenario where the dependence and or rank k is actually necessary:

My ideas

- Sample points from the unit circle, i.e $\cos(\theta)^2 + \sin(\theta)^2 = 1$ then
$$x = \sqrt{1 - \sin(\theta)^2} \text{ or } -\sqrt{1 - \sin(\theta)^2}$$
i.e sometihng can only be expressed with a rank 2 component?
- we could look for circuits, the goal is then no longer to find N independent features, but suppose we have input groups [0,1,2], [2,3,4], [3,4,5] then we want one component that activates for the first group, one for the second and one for the third

Remark 6:
I added the gnan plots to wandB remember to show them

Remark 7:
The per feature construction with only 1 feature is rather boring, and does not utilize the expressiveness at all.

Remark 8:
I dont think its a code problem, as i can reproduce equivalent results to their findings. Im just not able to beat them. We need to discuss experiments where the k dimensionality and dependence actually matters.

4.5.2 Per-dimension ablation and implicit rank selection (Hypothesis: ask lukas)

Following the extension to rank-k subcomponents, and the alternative formulation of the GNAN, the output $g_c^l \in [0,1]^k$ is a per feature importance value, and thus the masking can be applied independently to each feature. The intuition behind this is that if a mechanism captures a 1 dimensional feature, it would only need of the k dimensions to be active.
Likewise if this actually ends up working, k can be considered an upper bound or a so called "capacity" of a mechanism, it also follows nicely with the descriptive length principle found in the appendix of APD.

To determine the effective rank of each subcomponent, the stochastic masking becomes:
$$[m_c^l(x,r)]_j = [g_c^l(x)]_j + (1 - [g_c^l(x)]_j) r_{c,j}^l, \qquad r_{c,j}^l \sim U(0,1)$$
for each j in $1, \ldots, k$
Notably this treats each feature independently, which may not be desirable, as such we could modify it ever so slightly:

$$[m_c^l(x,r)]_j = [g_c^l(x)][g_c^l(x)]_j + (1 - [g_c^l(x)][g_c^l(x)]_j) r_{c,j}^l, \qquad r_{c,j}^l \sim U(0,1)$$
The signal becomes an importance of two factors, if the component is important and both features are important, it remains active, if only one feature is important, the other will be pushed to zero

The masked weight matrix becomes:
$$W'^l(x,r) = \sum_{c=1}^{C} U_c^l \operatorname{diag}\big(\mathbf{m}_c^l(x,r)\big)\, V_c^{l\top}$$
Where the diagonal matrix is the per-feature mask for subcomponent c.

The importance minimality loss penalises all k dimensions of all subcomponents toward zero:
$$\mathcal{L}_{\text{import}} = \sum_{l=1}^{L} \sum_{c=1}^{C} \sum_{j=1}^{k} \big|[g_c^l(x)]_j\big|^p$$

This follows the same logic by which SPD selects the number of active subcomponents: the minimality loss drives unused components toward zero, and only those that are genuinely required to reconstruct the target model's output survive.

This makes k a soft upper bound rather than a sensitive hyperparameter. In practice, k can be set conservatively large and the effective rank of each subcomponent is learned from the data.

5 Experiments

5.1 Reproducing SPD on independent features

5.2 Vanilla SPD on dependent features, the contrast experiment, two subsections, TMS sync and geometric model side by side.

5.3 Attention + softmax on TMS, Geometry

5.4 Extending rank k on Geometry, where k matters

¡ss

5.5 Toy model of superposition

5.5.1 Paired subcomponents

By engels et al, two features are irreducible if they always co-occur, as per this definition, two features that are always seen together should thus be treated as a single unit, this is something that the original SPD method does not do, it instead splits them into to seperate subcomponents.

5.6 Toy model of geometry

Again if we extend the line of engels, we would expect the simplex to compose into a singular unit, in this plot we would expect one component to entirely capture the simplex, even though the individual features in themselves also form valid structures, like the vertices themselves

5.7 Toy model with relu

5.8 Smallest possible LM

6 Discussion

7 References

8 Other things to consider?

Minimal descriptive length?

8.1 That some features are multidimensional

Think back to SAE, that learn a dictionary of features, then find hte linear combination of those features, however that this fails for cyclical features, as its cheaper to learn a single feature, than learn a direction and a periodic function. Now for your experiment, the dream scenario is that we allowe a multi dimensional component, and that this can learn the cyclical feature, i.e a weekly cycle or modular arithmatic either do this or extend TMS to be multi-dimensional, show if it works better with ur gnn or not, likewise we need to show that through individual masking of features, we can teach a component to kill one of its features, if it is better explained by a rank 1 component than rank 2.

[BBS25] Lucius Bushnaq, Dan Braun, and Lee Sharkey. *Stochastic Parameter Decomposition*. 2025. arXiv: 2506.20790 [cs.LG]. URL: https://arxiv.org/abs/2506.20790.

[Bra+25] Dan Braun et al. *Interpretability in Parameter Space: Minimizing Mechanistic Description Length with Attribution-based Parameter Decomposition*. 2025. arXiv: 2501.14926 [cs.LG]. URL: https://arxiv.org/abs/2501.14926.

[Cha+25] David Chanin et al. *A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders*. 2025. arXiv: 2409.14507 [cs.CL]. URL: https://arxiv.org/abs/2409.14507.

[Cun+23] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023. arXiv: 2309.08600 [cs.LG]. URL: https://arxiv.org/abs/2309.08600.

[Lea+25] Patrick Leask et al. *Sparse Autoencoders Do Not Find Canonical Units of Analysis*. 2025. arXiv: 2502.04878 [cs.LG]. URL: https://arxiv.org/abs/2502.04878.